



Titanic - Machine Learning from Disaster

Nguyễn Dương Bảo Trân, Trương Thị Huyền Trang, Giang Hào Tường, Lê Thị Thủy Tiên

Đại Học Sài Gòn, Việt Nam

Tóm tắt

Báo cáo này trình bày quy trình xây dựng mô hình học máy nhằm dự đoán khả năng sống sót của hành khách trên tàu Titanic dựa trên dữ liệu do Kaggle cung cấp. Nghiên cứu tập trung vào việc khai thác dữ liệu, xử lý giá trị thiếu, tạo đặc trưng mới (feature engineering) và đánh giá hiệu năng mô hình để tìm ra phương pháp dự đoán tối ưu. Nhiều mô hình học máy đã được thử nghiệm, bao gồm *Logistic Regression*, *Support Vector Machine*, *Random Forest*, *XGBoost*, *Gradient Boosting* và *Voting Classifier*. Kết quả cho thấy *Voting Classifier (Soft Voting)* đạt hiệu năng tốt nhất với độ chính xác 0.8492 và F1-score trung bình 0.82, nhờ khả năng kết hợp ưu điểm của các mô hình tuyến tính và phi tuyến. Phân tích kết quả chỉ ra rằng các yếu tố giới tính (Sex), hạng vé (Pclass) và giá vé (Fare) có ảnh hưởng mạnh nhất đến khả năng sống sót của hành khách. Nghiên cứu đồng thời nhấn mạnh vai trò quan trọng của *feature engineering* và tinh chỉnh siêu tham số (**hyperparameter tuning**) trong việc nâng cao hiệu quả mô hình học máy.

Keywords: Titanic, Survival Prediction, Logistics Regression, XGBoost, Gradient Boosting, Super Vector Machine, Random Forest, Voting Classifier, Feature Engineering.

1. Giới thiệu

Thảm họa chìm tàu Titanic năm 1912 là một trong những tai nạn hàng hải bi thảm nhất lịch sử, gây hậu quả khiến cho hơn 1500 người đã thiệt mạng[1]. Dữ liệu hành khách được lưu trữ đầy đủ và phản ánh rõ mối quan hệ giữa các yếu tố nhân khẩu học với khả năng sống sót đã khiến *bộ dữ liệu Titanic* trở thành một trong những tập dữ liệu kinh điển và có giá trị nhất trong lĩnh vực phân tích dữ liệu và học máy (Machine Learning)[2].

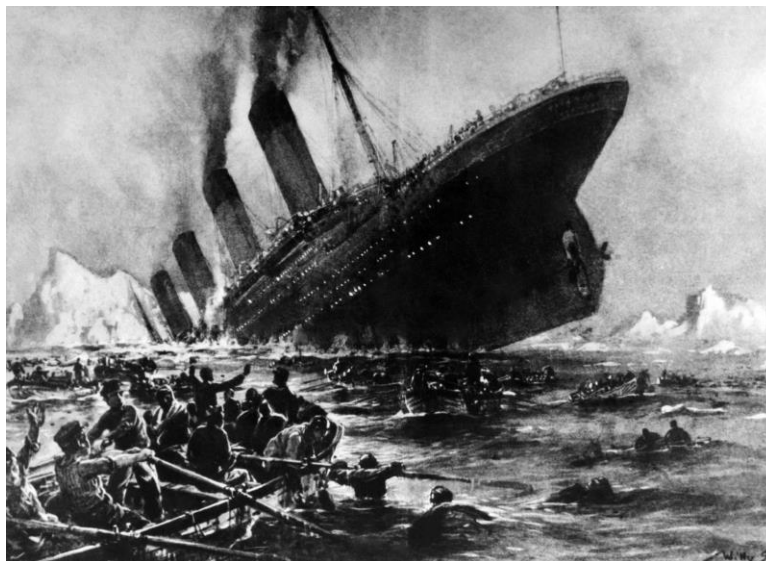


Fig. 1: Thảm họa chìm tàu Titanic

Trong những nghiên cứu ban đầu về dữ liệu Titanic, phần lớn các nhà nghiên cứu chỉ sử dụng các phương pháp truyền thống như *phân tích mô tả*, *hồi quy tuyến tính* hay *thăm định giả thuyết* nhằm

tìm hiểu các mối quan hệ giữa các đặc trưng như Sex, Pclass hay Age với khả năng sống sót. Tuy nhiên, những phương pháp này giả định mối quan hệ giữa các biến là tuyến tính và độc lập, nên khó phát hiện các tương tác phi tuyến phức tạp, dẫn đến *hiệu quả dự đoán còn hạn chế*[2].

Bộ dữ liệu Titanic chứa nhiều giá trị thiếu (đặc biệt ở *Age* và *Cabin*), mất cân bằng nhẹ giữa hai nhóm sống sót và kết hợp cả biến định tính lẫn định lượng, khiến việc xử lý và mô hình hóa trở nên phức tạp[3]. Vì vậy các phương pháp thống kê truyền thống không còn phù hợp để khai thác sâu dữ liệu này.

Từ thực tế đó, nhóm nghiên cứu đã áp dụng các mô hình học máy như Logistic Regression, Support Vector Machine (SVM), và Random Forest, đồng thời sử dụng phương pháp Voting (Soft Voting Classifier) để kết hợp ưu điểm của nhiều mô hình khác nhau, nhằm nâng cao độ chính xác dự đoán khả năng sống sót của hành khách. Cách tiếp cận sẽ tập trung vào ba nội dung chính: *Tiền xử lý dữ liệu*, *Tạo đặc trưng (feature engineering)* cuối cùng *Huấn luyện và đánh giá mô hình*[9].

Mục tiêu chính của nghiên cứu là xây dựng và đánh giá các mô hình máy học có khả năng dự đoán chính xác khả năng sống sót của hành khách trên tàu Titanic, đồng thời xác định những yếu tố có ảnh hưởng lớn nhất đến kết quả này. Qua đó góp phần củng cố những hiểu biết về quy trình xây dựng mô hình học máy hoàn chỉnh, từ khám phá và xử lý dữ liệu (EDA & preprocessing) cho đến đánh giá và phân tích kết quả.

2. Phương pháp đề xuất

2.1. Tổng quan vấn đề

Dựa trên bộ dữ liệu Titanic, chúng tôi nghiên cứu và đưa ra dự đoán sự sống còn của các hành khách với những thách thức đáng kể. Với thông tin của các hành khách trên chuyến tàu Titanic, biến mục tiêu là “Survived” thể hiện tình trạng sống sót của mỗi hành khách, trong đó giá trị bằng 1 là biểu thị “sống sót” và bằng 0 là biểu thị “Không sống sót”. Cho thấy đây là bài toán phân loại nhị phân (Binary Classification).

Dữ liệu đầu vào là 2 tập huấn luyện có 891 mẫu và tập kiểm thử có 418 mẫu. Trong bộ dữ liệu này tồn tại một số thách thức đó là thiếu giá trị ở đặc trưng Age, Cabin, Embarked và mất cân bằng lớp với tỷ lệ không sống sót cao hơn chiếm khoảng 61.6% so với 38.4% sống sót. Mục tiêu dự đoán khả năng sống sót của các hành khách là các đặc trưng đầu vào bao gồm:

- PassengerId: Mã hành khách
- Survived: Biến mục tiêu (1 = “Sống sót”, 0 = “Không sống sót”)
- Pclass: Hạng vé (1 = Hạng nhất, 2 = Hạng hai, 3 = Hạng ba)
- Name: Tên hành khách
- Sex: Giới tính
- Age: Tuổi
- SibSp : số anh/chị/em ruột hoặc vợ/chồng đi cùng
- Parch : số cha/mẹ hoặc con đi cùng
- Ticket : mã vé
- Fare : giá vé
- Cabin : số phòng
- Embarked - cảng lên tàu (C = Cherbourg, Q = Queenstown, S = Southampton)

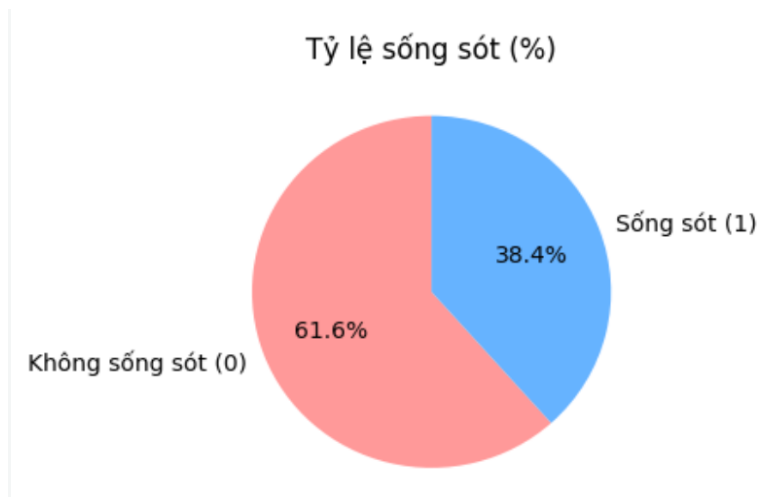


Fig. 2: Tỷ lệ sống sót

Trong nghiên cứu này, nhóm đã tiến hành thực nghiệm sáu mô hình học máy, bao gồm *Logistic Regression*, *Decision Tree*, *Random Forest*, *XGBoost*, *Support Vector Machine (SVM)*, *Gradient Boosting*. Bên cạnh đó sử dụng các chỉ số phổ biến trong bài toán nhị phân để đánh giá hiệu quả của các mô hình trên dự đoán khả năng sống sót đó là *Accuracy*, *Precision*, *Recall* và *F1-Score*.

2.2. Mô hình đề xuất

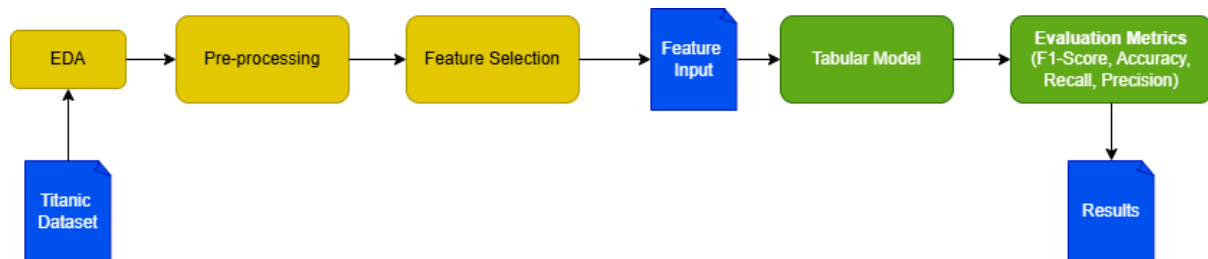


Fig. 3: Tổng quan phương pháp đề xuất

Nhiều nghiên cứu trước đây đã xem xét bài toán **dự đoán khả năng sống sót của hành khách trên tàu Titanic** nhằm so sánh hiệu quả của các thuật toán học máy khác nhau[9]. Các công trình này thường tập trung vào việc tối ưu hóa hiệu năng mô hình thông qua lựa chọn đặc trưng, chuẩn hóa dữ liệu và cân bằng giữa độ chính xác (*accuracy*) và khả năng tổng quát hóa (*generalization*).

Kế thừa và phát triển từ những hướng tiếp cận trước, **nhóm đề xuất mô hình Voting Classifier (Soft Voting)** — một phương pháp **ensemble learning** có khả năng kết hợp kết quả dự đoán của nhiều mô hình khác nhau nhằm nâng cao độ ổn định và hiệu quả tổng thể. Cụ thể, ba mô hình thành phần được lựa chọn bao gồm:

- **Logistic Regression.** Đại diện cho nhóm mô hình tuyến tính, có khả năng diễn giải tốt và hoạt động hiệu quả với dữ liệu chuẩn hóa.
- **Random Forest.** Mô hình cây quyết định dạng tập hợp (bagging), có khả năng xử lý các mối quan hệ phi tuyến và tương tác phức tạp giữa các đặc trưng.
- **Gradient Boosting:** mô hình boosting mạnh mẽ, tối ưu dần sai số dự đoán qua từng cây con, giúp tăng hiệu năng tổng thể.

Các mô hình trên được tinh chỉnh siêu tham số bằng **GridSearchCV (k-fold = 5)** để đạt cấu hình tối ưu nhất trước khi kết hợp trong Voting Classifier. Phương pháp **Soft Voting** được sử dụng để tính trung bình có trọng số của xác suất dự đoán, giúp khai thác đồng thời ưu điểm của từng thuật toán.

Mục tiêu của mô hình đề xuất là **tối đa hóa độ chính xác dự đoán khả năng sống sót** trong khi vẫn đảm bảo sự cân bằng giữa *Precision* và *Recall*, qua đó chứng minh tính hiệu quả của phương

pháp ensemble trong bài toán phân loại dữ liệu thực tế có đặc trưng đa dạng và không cân bằng như bộ dữ liệu Titanic.

2.3. Chi tiết triển khai

Dựa vào bộ dữ liệu Titanic, nhóm chúng tôi khám phá các mô hình học máy được dùng rất phổ biến trong bài toán phân lớp và hồi quy. Từ mô hình cơ bản *Logistic Regression* mô hình phức tạp hơn như *Super Vector Machine* và *XGBoost* được áp dụng trong nghiên cứu của nhóm.

Logistic Regression model. Đây là mô hình tuyến tính cơ bản, được sử dụng phổ biến trong các bài toán phân loại nhị phân. Với tập dữ liệu có nhiều giá trị thiếu và phân bố không cân bằng, nhóm đã tiến hành xử lý và chuẩn hóa dữ liệu cẩn thận trước khi huấn luyện mô hình. Xác suất một hành khách sống sót được tính theo công thức:

$$P(\text{Survived} = 1 | X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}} \quad (1)$$

Các hệ số β_i là hệ số của từng biến độc lập x_i cung cấp khả năng diễn giải định lượng về mức độ ảnh hưởng của từng biến độc lập đối với xác suất sống sót của hành khách. Ở biến giới tính (Sex) giá trị Female cho thấy khách hàng nữ có khả năng sống sót cao hơn so với nam giới, trong khi đó hạng vé (Pclass) phản ánh rằng hành khách ở hạng thấp hơn có xác suất sống sót thấp hơn.

Random Forest model. Mô hình được biết đến với sự kết hợp nhiều *Decision Tree* để tạo thành một mô hình mạnh. Việc áp dụng mô hình này cho bộ dữ liệu mang lại khả năng dự đoán chính xác hơn về việc giúp loại bỏ nhiễu và sai lệch trong dữ liệu.

XGBoost model. Chúng tôi đánh giá mô hình này là mô hình khá phức tạp nhưng lại là mô hình mạnh nhất trong việc dự đoán xác suất sống sót của hành khách. Không chỉ vậy, có khả năng khai thác mối quan hệ phức tạp giữa các đặc trưng của hành khách trên tàu.

Super Vector Machine (SVM) model. Chúng tôi sử dụng mô hình này để tìm ra một siêu phẳng tối ưu phân tách các mẫu dữ liệu thuộc hai lớp ($\text{Survived} = 1$ và $\text{Survived} = 0$) sao cho khoảng cách biên giữa siêu phẳng và các điểm gần nhất có thể giữa 2 lớp.

Gradient Boosting model. Đây là mô hình nhóm nghiên cứu cảm thấy mang lại hiệu quả vượt trội khi áp dụng vào bài toán phân loại với khả năng học phi tuyến nắm bắt quan hệ phức tạp giữa các đặc trưng như Pclass, Sex, Age, Fare. Giảm sai số dần theo vòng lặp khi mô hình được huấn luyện để sửa lỗi các cây trước, giúp cải thiện độ chính xác dần theo từng bước.

Accuracy. Dựa vào độ chính xác, chúng tôi lấy các mẫu *True Positive (TP)*, *True Negative (TN)*, *False Positive (FP)*, *False Negative (FN)* để thể hiện tỷ lệ mẫu của các mô hình dự đoán đúng.

$$\text{Accuracy} = \frac{(TP+TN)}{(TP+TN+FP+FN)} \quad (2)$$

Precision. Chỉ số này tính tỷ lệ mẫu dựa vào *True Positive (TP)* và *False Positive (FP)* đánh giá các mô hình để đo độ chính xác giảm thiểu số lượng dự đoán những trường hợp mô hình dự đoán “sống sót” không thực tế.

$$\text{Precision} = \frac{TP}{(TP + FP)} \quad (3)$$

Recall. Tương tự như *Precision* chỉ số này cũng lấy mẫu *True Positive* chỉ khác ở chỗ lấy mẫu của *False Negative* đo độ phủ để phát hiện các trường hợp tốt của hành khách còn sống sót nhưng có thể kèm theo với việc tăng số lượng dự đoán *False Positive (FP)*.

$$Recall = \frac{TP}{TP+FN} \quad (4)$$

F1-Score. Khác với các chỉ số ở trên, *F1-Score* là sự trung bình hài hòa giữa *Precision* và *Recall* giúp khả năng cân bằng những dữ liệu mất cân bằng như bộ dữ liệu Titanic. Giúp cho các mô hình có hiệu năng tốt và ổn định trong việc phân loại giữa 2 lớp “sống sót” - 1 và “không sống sót” - 0

$$F1 = 2 \times \frac{Precision+Recall}{Precision \times Recall} \quad (5)$$

2.4. Cấu trúc báo cáo

Báo cáo nghiên cứu được cấu trúc gồm năm phần chính, phản ánh toàn bộ quy trình từ phân tích vấn đề đến đánh giá kết quả và rút ra kết luận:

Phần 1 – Giới thiệu:

Phần mở đầu trình bày bối cảnh lịch sử và ý nghĩa thực tiễn của bài toán Titanic – một trong những bài toán kinh điển trong lĩnh vực học máy. Phần này làm rõ lý do lựa chọn đề tài, mục tiêu nghiên cứu, phạm vi triển khai và tầm quan trọng của việc dự đoán khả năng sống sót của hành khách dựa trên dữ liệu nhân khẩu học.

Phần 2 – Phương pháp đề xuất:

Phần này trình bày chi tiết các bước hình thành mô hình học máy, bao gồm:

- (2.1) Tổng quan vấn đề: Mô tả bài toán phân lớp nhị phân, cấu trúc dữ liệu đầu vào – đầu ra, thách thức của tập dữ liệu (giá trị thiếu, mất cân bằng lớp, nhiễu), và các tiêu chí đánh giá mô hình như Accuracy, F1-score, ROC-AUC.
- (2.2) Mô hình đề xuất: Giải thích lý do chọn các thuật toán học máy (Logistic Regression, Decision Tree, Random Forest, XGBoost, SVM, Gradient Boosting) và mô tả cơ chế kết hợp trong mô hình Voting Classifier.
- (2.3) Chi tiết triển khai: Trình bày quy trình kỹ thuật cụ thể gồm tiền xử lý dữ liệu (xử lý giá trị thiếu, mã hóa biến phân loại, chuẩn hóa), mở rộng đặc trưng (feature engineering), chia tập dữ liệu, tối ưu tham số (Grid Search), và đánh giá hiệu năng mô hình.
- (2.4) Cấu trúc báo cáo: Mục hiện tại – tóm tắt bố cục của toàn bộ nghiên cứu để giúp người đọc nắm bắt mạch triển khai của bài báo.

Phần 3 – Thực nghiệm:

Trình bày chi tiết các bước thực hiện, bao gồm tiền xử lý dữ liệu, trích xuất và chọn đặc trưng, huấn luyện và so sánh các mô hình. Mỗi thí nghiệm được minh họa bằng hình vẽ, bảng kết quả và phân tích kèm theo. Nhận xét được trình bày theo hướng từ tổng quan đến chi tiết, nhằm làm rõ hiệu quả của từng mô hình và tác động của các bước xử lý dữ liệu.

Phần 4 – Kết quả:

Tổng hợp và so sánh kết quả thực nghiệm của các mô hình trước và sau tinh chỉnh. Trình bày bảng so sánh độ chính xác (Accuracy), F1-score, ROC-AUC,... giữa các mô hình. Các kết quả tốt nhất được minh họa bằng biểu đồ trực quan (bar chart hoặc heatmap) kèm nhận xét lý giải tại sao mô hình đó đạt hiệu quả cao hơn.

Phần 5 – Kết luận và hướng phát triển:

Tổng kết các phát hiện chính của nghiên cứu, nêu bật ưu điểm và hạn chế của phương pháp hiện tại. Đồng thời đề xuất hướng phát triển trong tương lai, như mở rộng bộ dữ liệu, thử nghiệm các mô hình học sâu (Deep Learning) hoặc cải thiện kỹ thuật ensemble để tăng độ chính xác dự đoán.

Phần Tài liệu tham khảo:

Liệt kê toàn bộ các công trình, bài báo, và nguồn dữ liệu được trích dẫn trong nghiên cứu theo chuẩn IEEE.

3. Thực nghiệm và kết quả

Import các thư viện cần thiết và lấy dữ liệu từ tập train, test trên dữ liệu Titanic lấy từ <https://www.kaggle.com/competitions/titanic/data>

3.1. Phân tích khám phá dữ liệu (EDA)

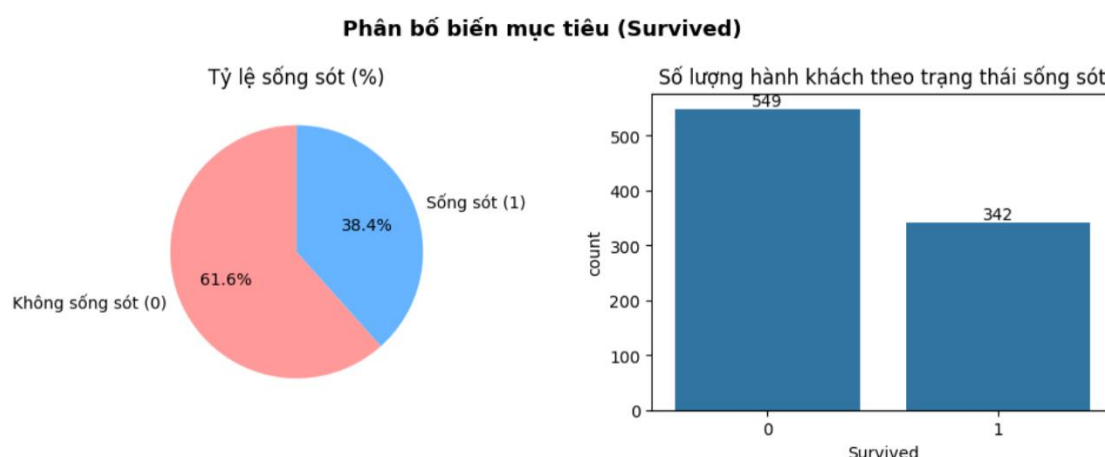


Fig. 4: Phân bố của biến mục tiêu *Survived*

Fig. 4 trên thể hiện phân bố của biến mục tiêu *Survived*, mô tả tình trạng sống sót của hành khách trên tàu *Titanic*. Biến này có hai giá trị 0 - “*Không sống sót*” và 1 - “*sống sót*”. Từ biểu đồ cột và biểu đồ tròn, có thể nhận thấy 549 hành khách (61.6%) - “*không sống sót*” và 342 hành khách (38.4%) - “*sống sót*”. Điều này cho thấy tỉ lệ không sống chiếm phần lớn gây ra dữ liệu mất cân bằng, ảnh hưởng đến hiệu suất của các mô hình học máy sự chênh lệch về lớp “*không sống sót*” nhiều hơn. Ở giai đoạn xử lý hoặc huấn luyện nhóm chúng tôi sẽ dùng kỹ thuật cân bằng dữ liệu (resampling) hoặc sử dụng trọng số lớp (class weight) để giảm thiểu ảnh hưởng này.

3.1.1. Phân bố của cột dạng số

3.1.1.1. Phân bố của biến Age

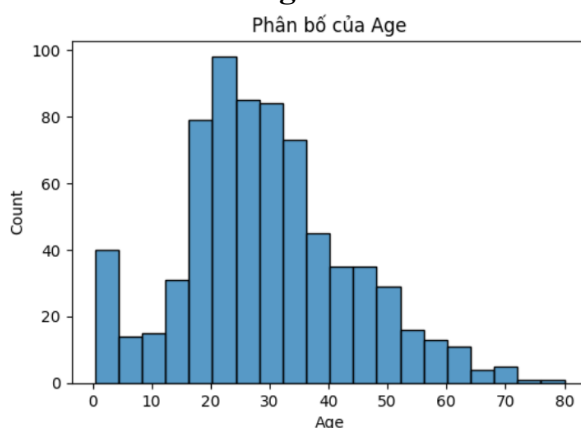


Fig. 5: Phân bố của biến *Age*

Dựa vào **Fig. 5**, phân bố độ tuổi của các hành khách trên tàu *Titanic* không đều và hơi lệch sang phải, chủ yếu là tập trung vào khoảng 20 - 40 tuổi. Cho thấy đây là nhóm tuổi chiếm đa số của

các hành khách trên tàu. Số lượng hành khách dưới 10 tuổi hoặc trên 60 tuổi tương đối ít, cho thấy phần lớn hành khách là người trưởng thành trẻ tuổi. Một số giá trị ngoại lai xuất hiện ở độ tuổi trên 70, tuy nhiên tần suất rất thấp.

Thấy rằng dữ liệu có xu hướng tập trung ở nhóm tuổi lao động, phản ánh thực tế rằng phần lớn hành khách Titanic là người đi làm hoặc di cư. Phân bố lệch phải có khả năng tồn tại một vài giá trị ngoại lai cần được xem xét trong quá trình tiền xử lý. Biến *Age* là biến liên tục có thể ảnh hưởng đến khả năng sống sót và thường được chia nhóm để các mô hình học máy khai thác hiệu quả hơn.

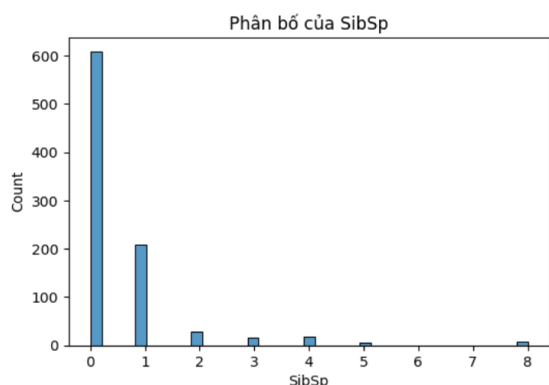


Fig. 6: Phân bố của biến *SibSp*

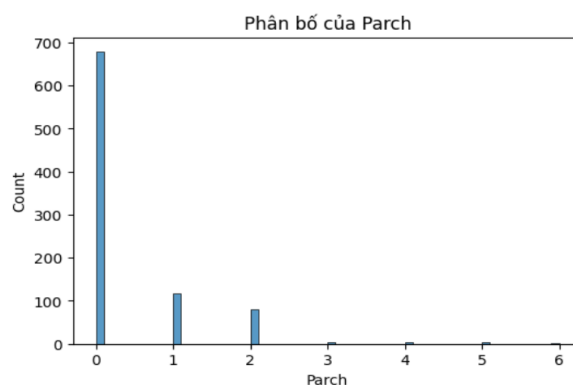


Fig. 7: Phân bố của biến *Parch*

3.1.1.2. Phân bố của các biến *SibSp* và *Parch*

Ở **Fig. 6** và **Fig. 7** cho thấy kết quả phần lớn hành khách có giá trị bằng 0 nghĩa là đa số hành khách đi một mình. Số lượng hành khách có từ 1–2 người thân đi cùng chiếm tỷ lệ nhỏ và các giá trị lớn hơn (từ 3 trở lên) hầu như rất hiếm. Phân bố của *SibSp* và *Parch* đều lệch mạnh về bên trái, tập trung chủ yếu ở giá trị thấp. Điều này phản ánh thực tế rằng phần đông hành khách đi tàu Titanic không đi cùng người thân, có thể do mục đích cá nhân như làm việc hoặc di cư.

Mặc dù phần lớn giá trị là 0, nhưng sự xuất hiện của một số hành khách đi cùng gia đình (giá trị 1–2) có thể ảnh hưởng đến hành vi và khả năng sống sót của họ. Đây là yếu tố cần xem xét trong các phân tích tương quan sau này.

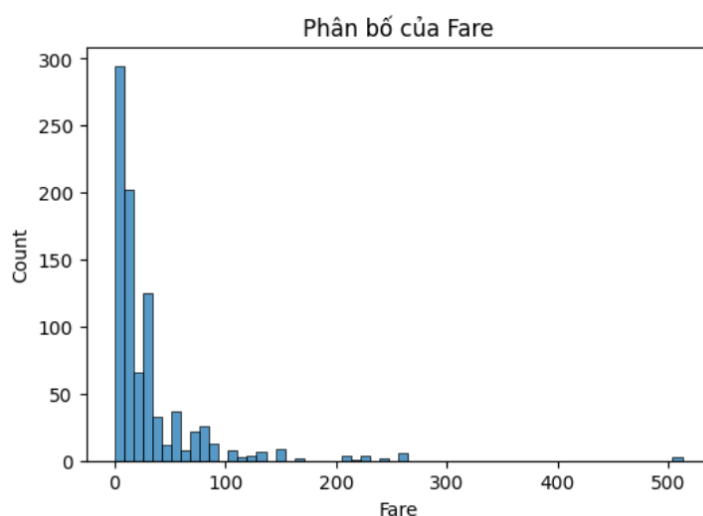


Fig. 8: Phân bố của biến *Fare*

3.1.1.3. Phân bố của biến Fare

Fig. 8 cho thấy biến *Fare* có phân bố lệch mạnh sang phải với phần lớn giá trị tập trung trong khoảng 0–100. Tuy nhiên, vẫn tồn tại một số giá trị ngoại lai cao (trên 300) cho thấy những hành khách trả giá vé đắt đỏ - thường là các hành khách ở hạng vé 1 ($Pclass = 1$).

Phân bố lệch phải cho thấy sự chênh lệch lớn về điều kiện kinh tế giữa các nhóm hành khách. Đa phần các hành khách có điều kiện tài chính trung bình hoặc thấp. Các giá trị ngoại lai (*outlier*) cần được xem xét trong quá trình tiền xử lý (có thể dùng log transform để giảm độ lệch). Biến *Fare* là một đặc trưng quan trọng, vì hành khách trả giá vé cao thường ở hạng vé cao hơn, có cơ hội sống sót cao hơn trong tai nạn.

3.1.2. Phân bố của cột phân loại

3.1.2.1. Phân bố của biến Pclass

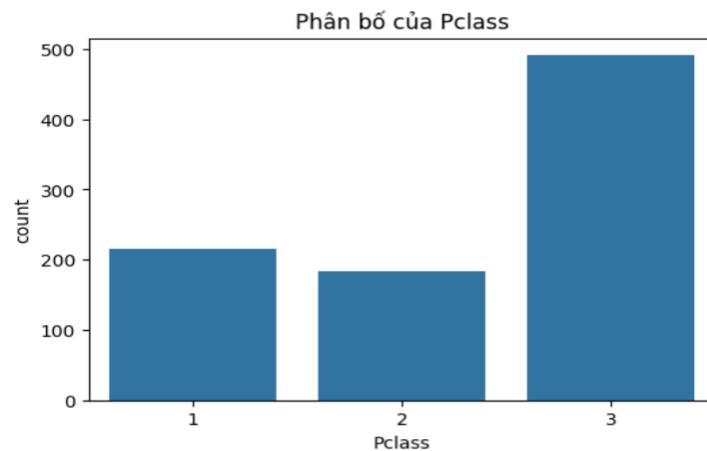


Fig. 9: Phân bố của biến *Pclass*

Ở **Fig. 9** thể hiện sự phân bố của ba hạng vé (*Pclass*). Kết quả hành khách hạng 3 chiếm tỷ lệ lớn nhất, trong khi hạng 1 và hạng 2 có số lượng thấp hơn. Hành khách hạng 3 chiếm đa số, phản ánh rằng lượng hành khách trên tàu Titanic thuộc tầng lớp trung lưu và bình dân. Hạng vé có thể liên quan mật thiết đến điều kiện sinh hoạt, vị trí cabin và cơ hội tiếp cận phương tiện cứu hộ, nên được xem là yếu tố dự đoán quan trọng cho biến mục tiêu *Survived*.

3.1.2.2. Phân bố của biến Sex

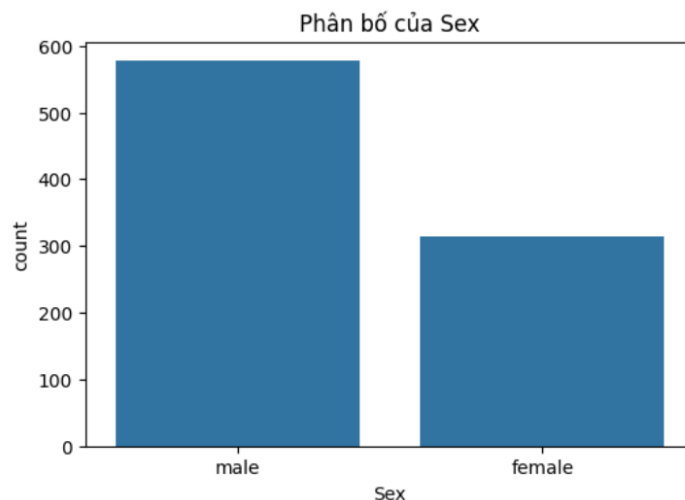


Fig. 10: Phân bố của biến *Sex*

Fig. 10 cho thấy biến *Sex* gồm hai giá trị male và female, trong đó nam giới chiếm phần lớn trong tập dữ liệu. Tuy nhiên, giới tính có thể đóng vai trò quan trọng trong khả năng sống sót do quy tắc “*women and children first*” được áp dụng khi xảy ra thảm kịch. Vì vậy biến *Sex* có tiềm năng là một đặc trưng phân loại mạnh trong mô hình dự đoán.

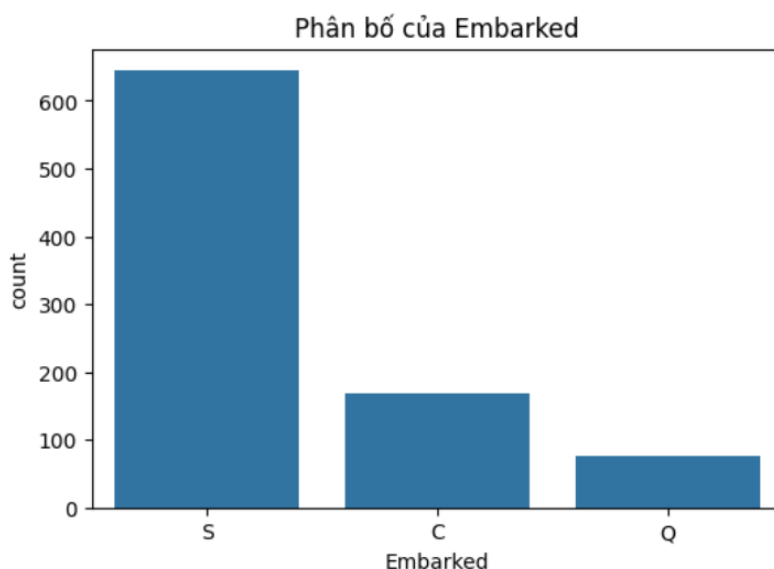


Fig. 11: *Phân bố của biến Embarked*

3.1.2.3. Phân bố của biến Embarked

Quan sát **Fig. 11** thể hiện phân bố của biến *Embarked* - nơi hành khách lên tàu. Biến này gồm ba giá trị S (*Southampton*), C (*Cherbourg*), Q (*Queenstown*). Cho thấy đa số hành khách lên tàu tại cảng Southampton (S) chiếm tỷ lệ lớn nhất trong 2 cảng còn lại. Tiếp đến là *Cherbourg* (C) và *Queenstown* (Q) với số lượng thấp đáng kể.

Cảng *Southampton* là điểm khởi hành chính của Titanic do đó cảng này chiếm ưu thế trong dữ liệu. Sự khác biệt về phân bố giữa ba cảng có thể liên quan đến đặc điểm địa lý, tầng lớp xã hội và giá vé của hành khách. Ví dụ, hành khách lên tàu tại *Cherbourg* thường thuộc tầng lớp khá giả hơn. Biến *Embarked* mặc dù chỉ có ba giá trị danh mục, nhưng có thể góp phần phản ánh nguồn gốc và điều kiện kinh tế của hành khách — các yếu tố gián tiếp ảnh hưởng đến khả năng sống sót.

3.1.3. Mối quan hệ của các biến đầu vào so với biến mục tiêu:

Hành khách có độ tuổi dao động trong khoảng 20–40 tuổi chiếm tỷ lệ cao nhất trong tập dữ liệu. Tuy nhiên, từ **Fig. 12** có thể thấy rằng nhóm trẻ em dưới 10 tuổi có tỷ lệ sống sót cao rõ rệt phản ánh chính sách ưu tiên phụ nữ và trẻ em trong quá trình cứu hộ khi thảm kịch xảy ra.

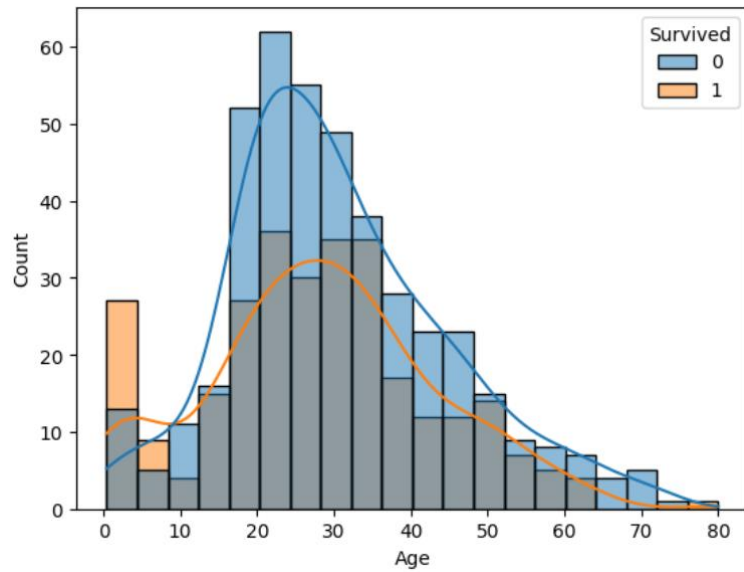


Fig. 12: Phân bố của Age so với Survived

Từ **Fig. 13**, ta thấy hành khách có mức giá vé (*fare*) cao thường có tỷ lệ sống sót lớn hơn. Điều này có thể liên quan đến hạng vé - những hành khách trả giá vé cao thường thuộc hạng ghế cao hơn, do đó họ được ưu tiên và có điều kiện an toàn hơn trong quá trình cứu hộ.

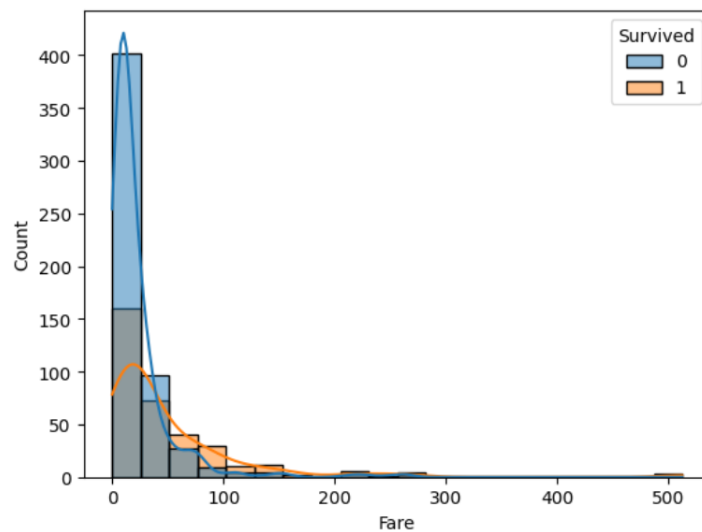


Fig. 13: Phân bố của Fare so với Survived

Từ **Fig. 14** và **Fig. 15**, ta thấy hành khách đi một mình có tỷ lệ sống sót thấp hơn so với những người đi cùng gia đình (khoảng 1–2 người). Việc có người thân đi cùng có thể giúp họ hỗ trợ lẫn nhau và tăng khả năng được cứu hộ trong tình huống khẩn cấp.

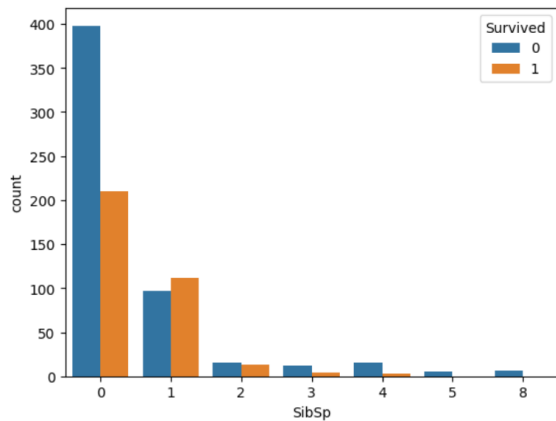


Fig. 14: Phân bố của SibSp so với Survived

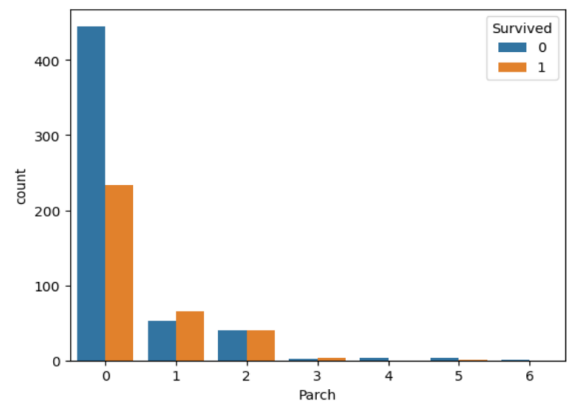


Fig. 15: Phân bố của Parch so với Survived

Pclass: Biểu đồ cho thấy hành khách ở hạng vé 1 ($Pclass = 1$) có tỷ lệ sống sót cao nhất, trong khi phần lớn hành khách ở hạng vé 3 lại có tỷ lệ tử vong cao hơn. Sự khác biệt này phản ánh sự bất bình đẳng về điều kiện và cơ hội tiếp cận phương tiện cứu hộ giữa các hạng vé. Hành khách hạng 1 thường có vị trí *cabin* ở tầng trên, gần boong tàu và xuống cứu sinh hơn, đồng thời nhận được ưu tiên cao hơn trong quá trình sơ tán. Ngược lại, hành khách hạng 3 ở khu vực dưới boong, xa lối thoát hiểm, dẫn đến khả năng sống sót thấp hơn.

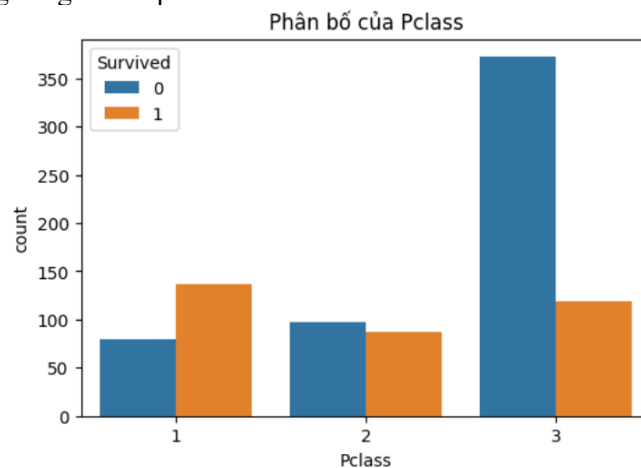


Fig. 16: Phân bố của Pclass so với Survived

Sex: Mặc dù số lượng hành khách nam chiếm đa số trên tàu Titanic, nhưng tỷ lệ sống sót của nữ giới lại cao hơn rõ rệt phản ánh chính sách ưu tiên cứu hộ phụ nữ và trẻ em ("*women and children first*") được áp dụng trong quá trình sơ tán.

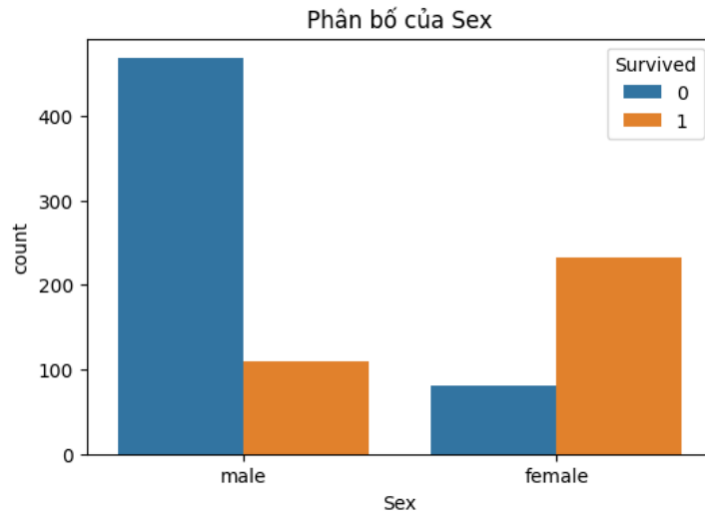


Fig. 17: *Phân bố của Sex so với Survived*

Embarked: Hành khách lên tàu tại cảng *Cherbourg* (C) có tỷ lệ sống sót cao hơn so với những người lên tàu tại cảng *Southampton* (S). Sự khác biệt này có thể xuất phát từ đặc điểm xã hội và kinh tế của hành khách tại từng cảng. Cụ thể, hành khách lên tàu tại *Cherbourg* thường thuộc tầng lớp khá giả, nhiều người trong số họ ở hạng vé 1 hoặc 2, do đó có điều kiện tốt hơn và được tiếp cận khu vực cứu hộ sớm hơn. Ngược lại, phần lớn hành khách xuất phát từ *Southampton* thuộc hạng vé 3, nên tỷ lệ sống sót thấp hơn.

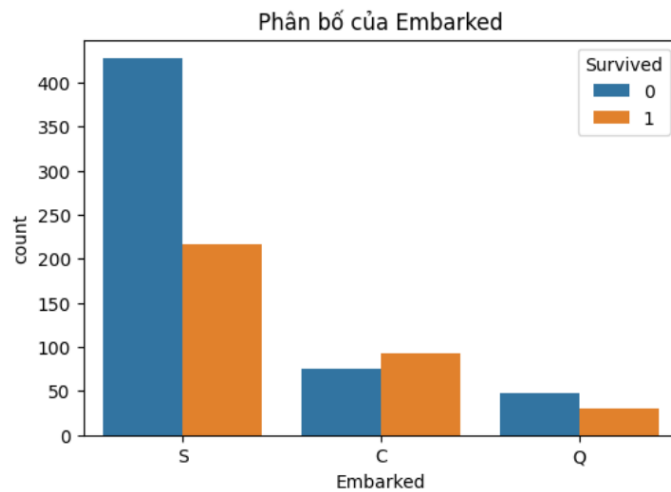


Fig. 18: *Phân bố của Embarked so với Survived*

Từ các biểu đồ trực quan, có thể thấy các yếu tố giới tính (*Sex*), hạng vé (*Pclass*) và giá vé (*Fare*) là những đặc trưng có tác động mạnh nhất đến khả năng sống sót của hành khách, trong khi các đặc trưng còn lại đóng vai trò bổ trợ.

3.1.4. Giá trị thiếu

Trong tập **train**, một số cột có chứa giá trị thiếu : **Age** có 177 giá trị thiếu, **Cabin** có 687 giá trị thiếu, **Embarked** có 2 giá trị thiếu.

```

===== Checking Null =====
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age             177
SibSp            0
Parch           0
Ticket           0
Fare             0
Cabin           687
Embarked         2
dtype: int64
--> Có giá trị Null: True

```

Fig. 19: Dữ liệu thiếu của tập train

Trong tập **test**, các cột có giá trị thiếu là Age có 86 giá trị thiếu, Fare có 1 giá trị thiếu, **Cabin** có 327 giá trị thiếu.

```

===== Checking Null =====
PassengerId      0
Pclass           0
Name             0
Sex              0
Age             86
SibSp            0
Parch           0
Ticket           0
Fare             1
Cabin           327
Embarked         0
dtype: int64
--> Có giá trị Null: True

```

Fig. 20: Dữ liệu thiếu của tập test

Như vậy, có thể thấy cột *Cabin* chứa tỷ lệ thiếu rất lớn ở cả hai tập dữ liệu, trong khi *Age* cũng có số lượng giá trị bị thiếu đáng kể. Điều này cho thấy cần phải xử lý dữ liệu thiếu phù hợp (như điền giá trị trung bình, trung vị hoặc tách đặc trưng “có/không có *Cabin*”) để tránh ảnh hưởng đến hiệu quả của mô hình.

3.1.5. Giá trị ngoại lai

Biểu đồ *boxplot* cho thấy nhiều giá trị ngoại lai xuất hiện ở phía giá vé cao, trong khi phần lớn hành khách có mức giá vé dưới 100. Điều này thể hiện phân phối lệch phải rõ rệt, đặc trưng bởi một số hành khách hạng nhất trả giá vé rất cao. Do đó, cần xem xét chuẩn hóa dữ liệu hoặc áp dụng phép biến đổi *logarit* (*log-transform*) để giảm ảnh hưởng của các giá trị ngoại lai này khi huấn luyện mô hình.

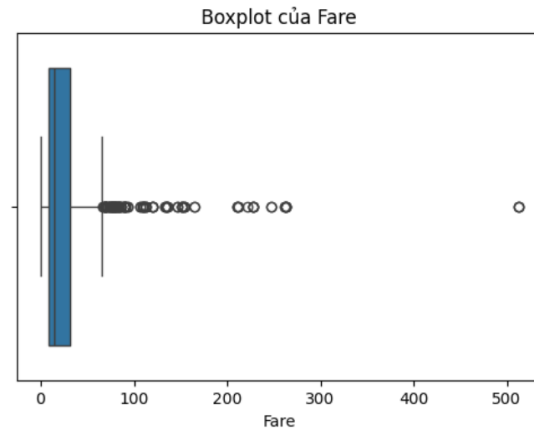


Fig. 21: Boxplot của biến Fare

Dựa trên *boxplot*, phân phối của độ tuổi khá đều, hầu như không xuất hiện nhiều giá trị ngoại lai rõ rệt. Phần lớn hành khách nằm trong khoảng 20–40 tuổi, đây là nhóm tuổi phổ biến nhất trong dữ liệu. Vì vậy, có thể giữ nguyên đặc trưng này, chỉ cần xử lý các giá trị thiếu trước khi đưa vào mô hình.

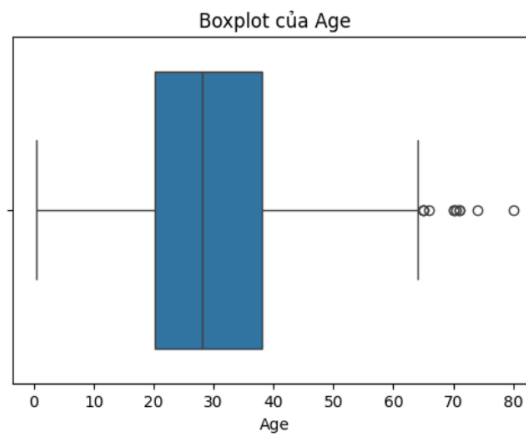


Fig. 22: Boxplot của biến Age

3.2. Tiền xử lý dữ liệu (Preprocessing)

Dữ liệu gốc được lấy từ bộ Titanic Dataset gồm hai tệp *train.csv* và *test.csv*. Trước khi tiến hành huấn luyện, dữ liệu được xử lý và làm sạch theo các bước sau:

3.2.1. Làm sạch dữ liệu

Một số cột không mang ý nghĩa dự đoán hoặc chứa thông tin không định lượng bị loại bỏ, chỉ giữ lại các cột Input và Output:

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Cabin	Embarked
0	0	3	male	22.0	1	0	7.2500	NaN	S
1	1	1	female	38.0	1	0	71.2833	C85	C
2	1	3	female	26.0	0	0	7.9250	NaN	S
3	1	1	female	35.0	1	0	53.1000	C123	S
4	0	3	male	35.0	0	0	8.0500	NaN	S

Fig. 23: Dữ liệu gốc từ tập train

Cột Cabin xuất hiện giá trị 'U' 687 lần, đại diện cho các ô bị thiếu thông tin. Việc thay NaN bằng 'U' giúp giữ lại đặc trưng Cabin mà không cần loại bỏ cột.

```

Tập test sau khi điền missing values
Age IsNa: 0
Age Median: 27.0
count    418.000000
mean      35.560845
std       55.856972
min        0.000000
25%        7.895800
50%       14.454200
75%       31.471875
max       512.329200
Name: Fare, dtype: float64
----- Cabin
(array(['A11', 'A18', 'A21', 'A29', 'A34', 'A9', 'B10', 'B11', 'B24',
       'B26', 'B36', 'B41', 'B45', 'B51 B53 B55', 'B52 B54 B56',
       'B57 B59 B63 B66', 'B58 B60', 'B61', 'B69', 'B71', 'B78', 'C101',
       'C105', 'C106', 'C116', 'C130', 'C132', 'C22 C26', 'C23 C25 C27',
       'C28', 'C31', 'C32', 'C39', 'C46', 'C51', 'C53', 'C54', 'C55 C57',
       'C6', 'C62 C64', 'C7', 'C78', 'C80', 'C85', 'C86', 'C89', 'C97',
       'D', 'D10 D12', 'D15', 'D19', 'D21', 'D22', 'D28', 'D30', 'D34',
       'D37', 'D38', 'D40', 'D43', 'E31', 'E34', 'E39 E41', 'E45', 'E46',
       'E50', 'E52', 'E60', 'F', 'F E46', 'F E57', 'F G63', 'F2', 'F33',
       'F4', 'G6', 'U'], dtype=object),
 array([ 1,  1,  1,  1,  2,  1,  1,  1,  1,  1,  1,  1,  2,
         1,  1,  3,  1,  1,  1,  1,  2,  1,  1,  2,  1,
         1,  1,  2,  1,  2,  1,  1,  1,  1,  1,  2,  2,
         1,  1,  2,  2,  1,  1,  2,  1,  1,  1,  1,  1,
         1,  1,  1,  1,  1,  1,  1,  1,  2,  1,  1,  1,
         1,  1,  1,  1,  1,  1,  1,  1,  1,  2,  1,  327]))

```

Fig. 26: Tập test sau khi điền thiếu và kết quả Cabin

Sau khi điền trung vị cho Age (Median = 27.0), không còn giá trị thiếu (Age IsNa: 0). Phân phối Fare hợp lý trong khoảng 7.89–14.45, chứng tỏ việc điền median theo Pclass chính xác. Cột Cabin đã được chuẩn hóa, các ô trống được thay bằng giá trị đại diện để giữ lại đặc trưng Cabin.

3.2.3. Mã hóa biến phân loại

Các cột dạng chuỗi được mã hóa về dạng số để mô hình có thể xử lý: Sử dụng *LabelEncoder* cho các cột *Sex*, *Cabin*, và *Embarked*.

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Cabin	Embarked
0	0	3	1	22.0	1	0	7.2500	147	2
1	1	1	0	38.0	1	0	71.2833	81	0
2	1	3	0	26.0	0	0	7.9250	147	2
3	1	1	0	35.0	1	0	53.1000	55	2
4	0	3	1	35.0	0	0	8.0500	147	2

Fig. 27: Các cột chuỗi được mã hóa về dạng số

- + Cột Sex: Female = 0, Male = 1
- + Cột Cabin: Mã cabin (A10, B20, C85, D35, ..., U) được LabelEncoder sắp xếp theo thứ tự chữ cái (A → U) và gán số thứ tự tương ứng. Ví dụ: A10 → 0, B20 → 12, C85 → 81, U → 147)
- + Cột Embarked: C = 0, Q = 1, S = 2.

3.2.4. Chuẩn hóa dữ liệu

Các biến liên tục như Age và Fare được chuẩn hóa bằng MinMaxScaler để đưa các giá trị về cùng thang đo [0, 1], giúp giảm ảnh hưởng của các giá trị lớn và tăng tốc độ hội tụ của mô hình.

$$z = \frac{x - \min(x)}{\max(x) - \min(x)}$$

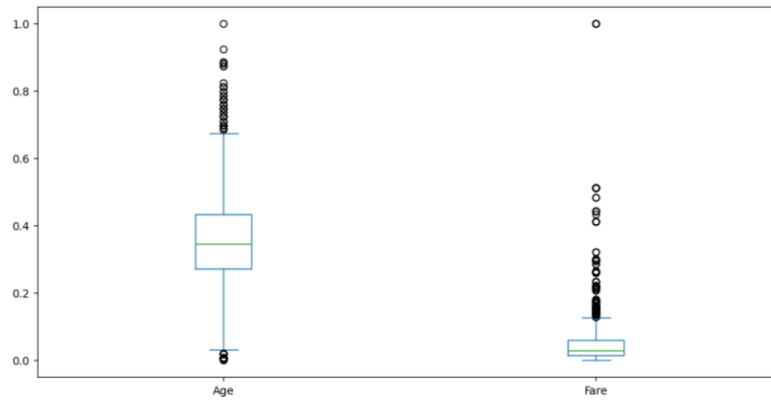


Fig. 28: Phân phối và ngoại lệ của Age và Fare sau khi chuẩn hóa

Biểu đồ *boxplot* cho thấy phân phối và ngoại lệ của Age và Fare sau khi chuẩn hóa giá trị Age phân bố tập trung quanh trung vị, có vài giá trị ngoại lệ (tuổi cao). Giá trị Fare: lệch phải, nhiều giá trị thấp và vài vé giá rất cao.

3.2.5. Lưu dữ liệu sau tiền xử lý

Sau khi hoàn tất mã hóa và chuẩn hóa, các tập dữ liệu X_{train} , X_{val} , y_{train} , y_{val} , và X_{test} được lưu vào thư mục features/ dưới định dạng .csv (hoặc .pkl) để tiện sử dụng cho các bước huấn luyện và đánh giá mô hình sau này.

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Cabin	Embarked
0	0	3	1	0.271174	1	0	0.014151	147	2
1	1	1	0	0.472229	1	0	0.139136	81	0
2	1	3	0	0.321438	0	0	0.015469	147	2
3	1	1	0	0.434531	1	0	0.103644	55	2
4	0	3	1	0.434531	0	0	0.015713	147	2

```

+ x_train:
[[[0.00000000e+00 3.00000000e+00 1.00000000e+00 2.71173662e-01
  1.00000000e+00 0.00000000e+00 1.41510576e-02 1.47000000e+02]
 [1.00000000e+00 1.00000000e+00 0.00000000e+00 4.72229203e-01
  1.00000000e+00 0.00000000e+00 1.39135735e-01 8.10000000e+01]
 [1.00000000e+00 3.00000000e+00 0.00000000e+00 3.21437547e-01
  0.00000000e+00 0.00000000e+00 1.54685698e-02 1.47000000e+02]
 [1.00000000e+00 1.00000000e+00 0.00000000e+00 4.34531289e-01
  1.00000000e+00 0.00000000e+00 1.03644297e-01 5.50000000e+01]
 [0.00000000e+00 3.00000000e+00 1.00000000e+00 4.34531289e-01
  0.00000000e+00 0.00000000e+00 1.57125536e-02 1.47000000e+02]]]
+ y_train:
[2. 0. 2. 2. 2.]

df_minmax.shape
(891, 9)

```

Fig. 29: Dữ liệu train sau khi chuẩn hóa

Bảng thể hiện tập huấn luyện df_minmax sau khi chuẩn hóa *MinMaxScaler*. Các đặc trưng số (Age, Fare) được đưa về thang [0,1], các biến phân loại (Sex, Cabin, Embarked) đã được mã hóa dạng số. x_{train} và y_{train} là ma trận đặc trưng và nhãn mục tiêu dùng cho quá trình huấn luyện. Kích thước dữ liệu: (891, 9).

	Pclass	Sex	Age	SibSp	Parch	Fare	Cabin	Embarked
0	3	1	0.428248	0	0	0.015282	147	1
1	3	0	0.585323	1	0	0.013663	147	2
2	2	1	0.773813	0	0	0.018909	147	1
3	3	1	0.334004	0	0	0.016908	147	2
4	3	0	0.271174	1	1	0.023984	147	2

```

+ x_test:
[[3.00000000e+00 1.00000000e+00 4.28248304e-01 0.00000000e+00
 0.00000000e+00 1.52815807e-02 1.47000000e+02]
[3.00000000e+00 0.00000000e+00 5.85322945e-01 1.00000000e+00
 0.00000000e+00 1.36630901e-02 1.47000000e+02]
[2.00000000e+00 1.00000000e+00 7.73812516e-01 0.00000000e+00
 0.00000000e+00 1.89087407e-02 1.47000000e+02]
[3.00000000e+00 1.00000000e+00 3.34003518e-01 0.00000000e+00
 0.00000000e+00 1.69080739e-02 1.47000000e+02]
[3.00000000e+00 0.00000000e+00 2.71173662e-01 1.00000000e+00
 1.00000000e+00 2.39836027e-02 1.47000000e+02]]

df_test_minmax.shape

(418, 8)

```

Fig. 30: Dữ liệu test sau khi chuẩn hóa

Tập kiểm tra *df_test_minmax* sau khi áp dụng cùng *scaler* và *encoder* từ tập huấn luyện. Chỉ gồm các đặc trưng đầu vào (không có nhãn *Survived*). Dữ liệu được chuẩn hóa đồng nhất giúp mô hình đánh giá chính xác hơn. Kích thước dữ liệu: **(418, 8)**.

3.2.6. Chia dữ liệu huấn luyện - kiểm định:

Trong quá trình huấn luyện, tập Train tiếp tục được chia thành Train (80%) và Validation (20%). Việc chia được thực hiện bằng *train_test_split()* của thư viện *scikit-learn*.

3.3. Trích xuất đặc trưng

Sau khi hoàn tất giai đoạn tiền xử lý dữ liệu (xử lý giá trị thiếu, mã hóa biến, chuẩn hóa thang đo), nhóm tiếp tục thực hiện bước trích xuất và mở rộng đặc trưng (*Feature Engineering*) nhằm khai thác thêm thông tin ẩn trong dữ liệu gốc.

Quá trình này giúp mô hình học máy hiểu sâu hơn về các mối quan hệ phi tuyến giữa các thuộc tính, từ đó cải thiện khả năng dự đoán.

Các đặc trưng mới được xây dựng bao gồm: *Cabin*, *Title (Name)*, *Family*, *AgeBin* và *FareBin* như sau:

3.3.1. Đặc trưng Cabin

Biến *Cabin* trong tập dữ liệu gốc chứa rất nhiều giá trị thiếu (khoảng 77% dữ liệu), nhưng vẫn mang ý nghĩa tiềm ẩn về vị trí của hành khách trên tàu.

Nhóm nghiên cứu đã xử lý cột này theo ba bước:

1. Thay thế các giá trị thiếu bằng ký hiệu 'U' (Unknown).
2. Tạo đặc trưng nhị phân *HasCabin* biểu thị việc hành khách **có thông tin cabin hay không** (1 nếu có, 0 nếu không).
3. Trích xuất ký tự đầu tiên của cabin làm đặc trưng *Deck*, thể hiện **tầng hoặc khu vực** nơi hành khách ở.

Biểu đồ cho thấy hành khách có cabin (*HasCabin* = 1) thường có **tỷ lệ sống sót cao hơn** so với nhóm không có thông tin cabin.

Tương tự, ở đặc trưng *Deck*, các tầng thuộc khu vực **A–C** có tỷ lệ sống sót cao hơn so với các tầng

thấp hơn (E–G), phản ánh thực tế rằng hành khách ở vị trí gần boong tàu có khả năng tiếp cận xuồng cứu sinh sớm hơn.

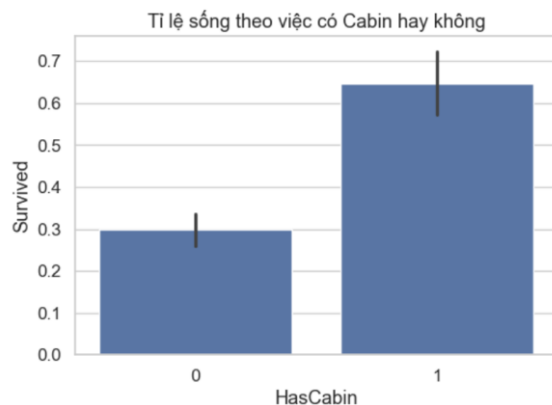


Fig. 31: Tỷ lệ sống theo việc có Cabin hay không

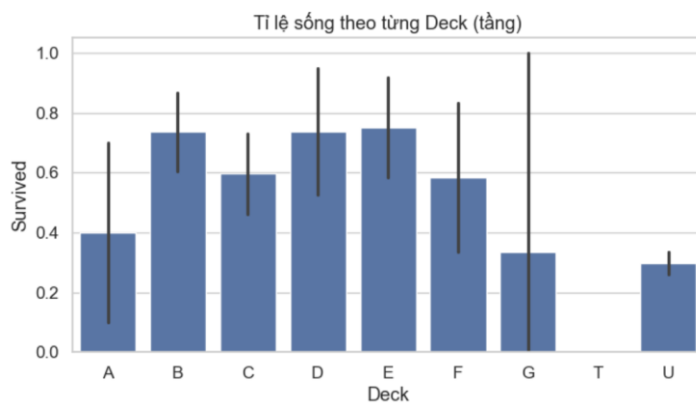


Fig. 32: Tỷ lệ sống theo từng Deck (tầng)

3.3.2. Đặc trưng danh xưng (Title) trích xuất từ Name

Từ trường Name, nhóm trích xuất danh xưng (Title) của hành khách như *Mr.*, *Mrs.*, *Miss.*, *Master.*, *Dr.*, v.v. bằng biểu thức chính quy (regular expression).

Các danh xưng hiếm như *Lady*, *Countess*, *Capt*, *Col*, *Don*, *Sir*,... được gộp lại thành nhóm "Rare", trong khi các danh xưng đồng nghĩa như "Mlle", "Ms" được chuẩn hóa thành "Miss", và "Mme" được đổi thành "Mrs".

Kết quả thống kê cho thấy các nhóm danh xưng phản ánh rõ giới tính và tầng lớp xã hội, ảnh hưởng trực tiếp đến khả năng sống sót.

Cụ thể, các danh xưng nữ giới (*Mrs.*, *Miss*) có **tỷ lệ sống cao hơn** đáng kể so với các danh xưng nam giới (*Mr.*, *Rev.*, *Dr.*,...).

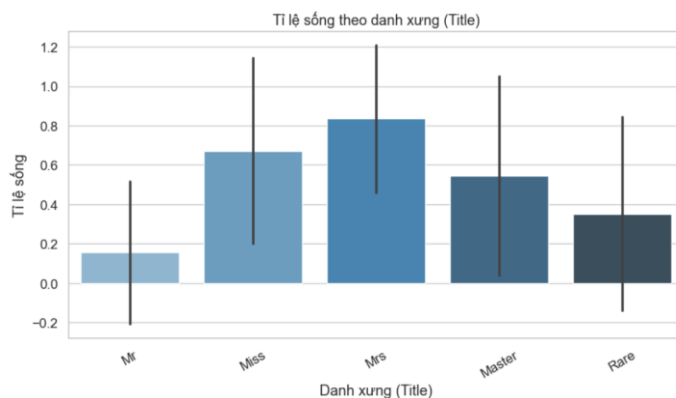


Fig. 33: Tỷ lệ sống theo danh xưng (Title)

3.3.3. Đặc trưng gia đình (Family)

Hai cột SibSp (số anh/chị/em hoặc vợ/chồng đi cùng) và Parch (số cha/mẹ hoặc con đi cùng) được kết hợp để tạo ra đặc trưng mới:

$$\text{FamilySize} = \text{SibSp} + \text{Parch} + 1$$

Đồng thời, một đặc trưng nhị phân khác được tạo ra:

$$\text{IsAlone} = \begin{cases} 1 & \text{nếu FamilySize} = 1 \text{ (đi một mình)} \\ 0 & \text{ngược lại} \end{cases}$$

Kết quả cho thấy hành khách **đi cùng gia đình nhỏ (2–4 người)** có tỷ lệ sống sót cao hơn, trong khi những người **đi một mình** hoặc **gia đình quá đông (>5 người)** lại có tỷ lệ thấp hơn.

Điều này gợi ý rằng **sự hỗ trợ giữa các thành viên trong gia đình** đóng vai trò quan trọng trong tình huống khẩn cấp.

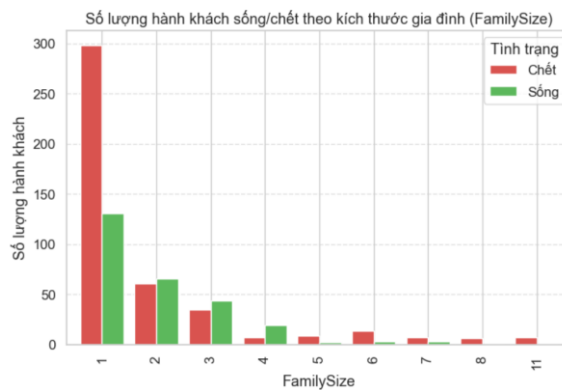


Fig. 34: Số lượng hành khách sống/chết theo kích thước gia đình (FamilySize)

3.3.4. Đặc trưng nhóm tuổi (AgeBin)

Thay vì sử dụng giá trị tuổi (Age) liên tục, nhóm chia thành 5 nhóm tuổi có ý nghĩa nhân khẩu học:

Nhóm	Khoảng tuổi	Mô tả
Child	0 – 11	Trẻ em
Teen	12 – 17	Thiếu niên
YoungAdult	18 – 29	Thanh niên
Adult	30 – 49	Người trưởng thành
Senior	>= 50	Trung niên và cao tuổi

Phân tích cho thấy nhóm **Child** có tỷ lệ sống sót cao nhất, tiếp theo là nhóm **Adult trẻ tuổi (YoungAdult)**, trong khi nhóm **Senior** có tỷ lệ sống thấp nhất. Điều này phù hợp với chính sách “phụ nữ và trẻ em trước” trong quá trình sơ tán.

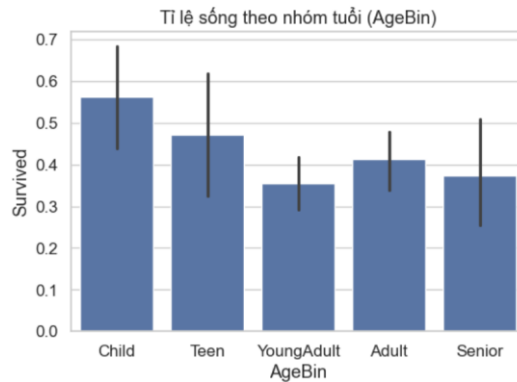


Fig. 35: Tỷ lệ sống theo nhóm tuổi (*AgeBin*)

3.3.5. Đặc trưng nhóm giá vé (*FareBin*)

Để giảm ảnh hưởng của các giá trị ngoại lệ và mô tả tốt hơn mối quan hệ phi tuyến, giá vé (*Fare*) được chia thành 4 nhóm dựa trên **phân vị (quartiles)** của dữ liệu huấn luyện:

Nhóm	Phân vị	Mức giá vé
Low	0 – 25%	Giá thấp
Mid	25 – 50%	Trung bình
High	50 – 75%	Cao
VeryHigh	75 – 100%	Rất cao

Kết quả cho thấy hành khách ở nhóm **High** và **VeryHigh** có khả năng sống sót cao hơn rõ rệt. Điều này phản ánh sự chênh lệch về điều kiện kinh tế giữa các hạng vé, khi hành khách hạng nhất thường có cabin tốt hơn và khả năng tiếp cận phương tiện cứu hộ cao hơn.

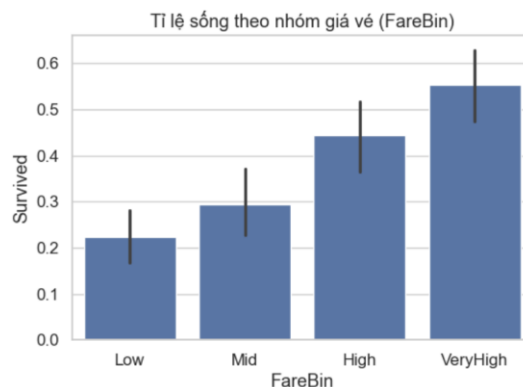


Fig. 36: Tỷ lệ sống theo nhóm giá vé (*FareBin*)

Các đặc trưng được trích xuất không chỉ giúp mô hình học được nhiều thông tin hơn mà còn **cải thiện khả năng tổng quát hóa**, đặc biệt với các mô hình tuyến tính vốn khó nắm bắt mối quan hệ phi tuyến giữa các biến.

Các đặc trưng mới như *Title*, *HasCabin*, *FamilySize*, *AgeBin*, và *FareBin* sau này đã chứng minh hiệu quả rõ rệt trong việc **tăng F1-score và Recall** của mô hình Logistic Regression và Voting Classifier.

3.4. Cài đặt thực nghiệm (Setup)

Các thí nghiệm được triển khai bằng ngôn ngữ Python, sử dụng các thư viện phổ biến trong lĩnh vực học máy bao gồm Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn và XGBoost. Toàn bộ quy trình được thực hiện trong môi trường lập trình thống nhất, với việc cố định hạt giống ngẫu nhiên (`random_state = 42`) nhằm đảm bảo tính tái lập kết quả trong quá trình huấn luyện.

Bộ dữ liệu Titanic: Machine Learning from Disaster được sử dụng để huấn luyện và đánh giá mô hình, bao gồm 891 mẫu huấn luyện và 418 mẫu kiểm thử. Tập dữ liệu được chia thành hai phần là tập huấn luyện (train) và tập kiểm thử (test) theo cấu trúc ban đầu của bộ dữ liệu Kaggle.

Các chỉ số được sử dụng để đánh giá hiệu năng mô hình bao gồm **Accuracy**, **Precision**, **Recall** và **F1-Score**. Cụ thể, **Accuracy** phản ánh tỷ lệ dự đoán chính xác tổng thể, **Precision** đo mức độ chính xác trong việc nhận diện các trường hợp sống sót, **Recall** đánh giá khả năng phát hiện đầy đủ các trường hợp thực sự sống sót, và **F1-Score** là trung bình điều hòa giữa Precision và Recall, giúp cân bằng giữa hai yếu tố này trong các tập dữ liệu có độ mất cân bằng lớp.

Trong giai đoạn huấn luyện, sáu mô hình học máy được thử nghiệm độc lập bao gồm **Logistic Regression**, **Decision Tree**, **Random Forest**, **XGBoost**, **Support Vector Machine (SVM)** và **Gradient Boosting**. Các mô hình được triển khai trong cùng điều kiện huấn luyện, sử dụng **Pipeline** để tích hợp các bước tiền xử lý, mã hóa đặc trưng (LabelEncoder), chuẩn hóa dữ liệu (MinMaxScaler), và mô hình phân loại nhằm đảm bảo tính nhất quán giữa các lần huấn luyện.

Sau khi xác định được ba mô hình có hiệu năng cao nhất là **Gradient Boosting**, **Random Forest** và **Logistic Regression**, nhóm nghiên cứu tiến hành xây dựng mô hình **Voting Classifier** theo phương pháp **Soft Voting**, nhằm kết hợp ưu điểm của các mô hình tuyến tính và phi tuyến. Cụ thể, dự đoán cuối cùng của mô hình được xác định dựa trên trung bình có trọng số của các xác suất dự đoán từ từng mô hình thành phần.

Để tối ưu hiệu suất, nhóm áp dụng **RandomizedSearchCV** với 30 lần thử nghiệm (`n_iter = 30`) trên không gian siêu tham số gồm các giá trị khác nhau của *learning_rate*, *max_depth*, *n_estimators*, và *C*. Quá trình tìm kiếm được đánh giá bằng **5-fold cross-validation** với thước đo **accuracy**. Mô hình tốt nhất thu được có độ chính xác trung bình trên tập xác thực là **0.8361**, thể hiện khả năng khái quát hóa tốt và ổn định khi kết hợp nhiều thuật toán.

Kết quả này cho thấy phương pháp kết hợp (ensemble) bằng **Soft Voting Classifier** giúp cải thiện hiệu năng tổng thể, đồng thời hạn chế rủi ro lệ thuộc vào một mô hình đơn lẻ, từ đó mang lại kết quả dự đoán tin cậy và nhất quán hơn trên bộ dữ liệu Titanic.

3.5. Kết quả (Result)

3.5.1. Mô hình trước khi trích xuất đặc trưng

	Model	Accuracy	Precision	Recall	F1
1	Gradient Boosting	0.837050	0.824784	0.709075	0.761776
5	Logistic Regression	0.797750	0.746311	0.693096	0.714517
2	Random Forest	0.794933	0.742044	0.693583	0.714095
4	XGBoost	0.787891	0.735838	0.680712	0.702736
3	Decision Tree	0.779441	0.711446	0.685534	0.695452
0	SVM	0.682727	0.662451	0.310426	0.417568

Fig. 37: Kết quả mô hình trước khi trích xuất đặc trưng

Trước khi thực hiện Feature Engineering, mô hình Gradient Boosting đạt hiệu năng cao nhất với Accuracy = 0.837 và F1 = 0.762, vượt trội so với các mô hình còn lại. Các mô hình tuyến tính như Logistic Regression đạt độ chính xác khá tốt (0.798) và cân bằng giữa Precision – Recall, trong khi SVM cho kết quả thấp nhất do hạn chế trong việc tách lớp phi tuyến trên tập dữ liệu gốc.

Nhìn chung, các mô hình cây (Decision Tree, Random Forest, Gradient Boosting, XGBoost) hoạt động hiệu quả hơn so với mô hình tuyến tính, cho thấy dữ liệu Titanic có đặc trưng phi tuyến tính rõ ràng.

3.5.2. Mô hình sau khi trích xuất với tham số cơ bản

	Model	Accuracy	Precision	Recall	F1
3	Gradient Boosting	0.824452	0.788679	0.709963	0.746410
0	Logistic Regression	0.807590	0.764969	0.695254	0.724622
5	SVM	0.794992	0.733019	0.706629	0.717304
2	Random Forest	0.796303	0.747804	0.687202	0.713555
4	XGBoost	0.789300	0.733354	0.693438	0.708794
1	Decision Tree	0.757023	0.673598	0.666265	0.668539

Fig. 38: Kết quả mô hình khi trích xuất đặc trưng

Nhận xét:

Sau khi áp dụng Feature Engineering (chuẩn hóa dữ liệu, log-transform Fare, tạo đặc trưng FamilySize, trích xuất Title từ Name, và mã hóa nhị phân Cabin/Embarked), hiệu năng của hầu hết các mô hình được cải thiện nhẹ về Recall và F1.

Đặc biệt, mô hình Logistic Regression và SVM cải thiện đáng kể F1-score so với baseline (tăng lần lượt khoảng +0.01 và +0.30), chứng tỏ các đặc trưng mới giúp các mô hình tuyến tính học tốt hơn mối quan hệ giữa các biến.

Tuy nhiên, Gradient Boosting vẫn là mô hình có hiệu năng tốt nhất với Accuracy = 0.8245 và F1 = 0.7464, cho thấy mô hình này vẫn giữ được khả năng tổng quát hóa tốt sau khi tinh chỉnh đặc trưng.

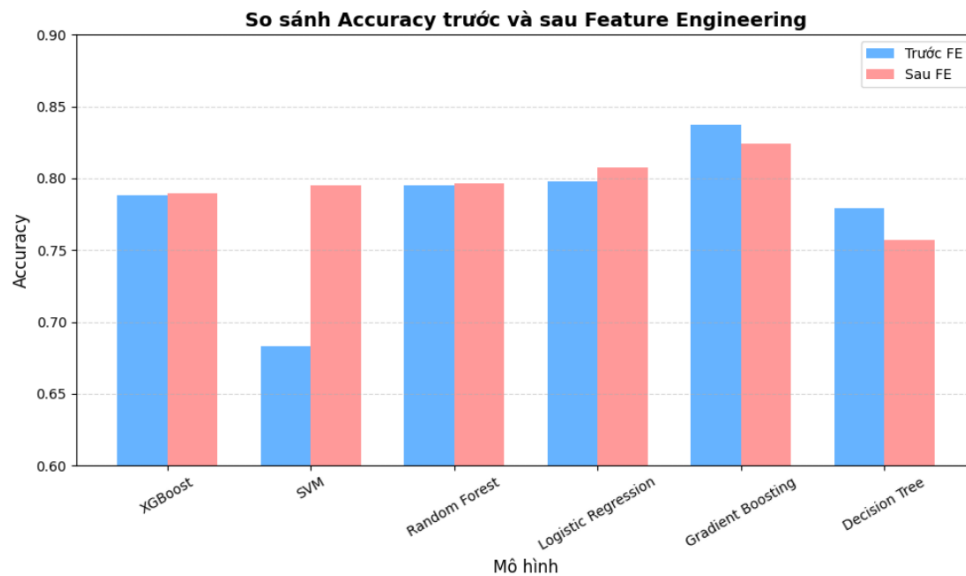


Fig. 39: So sánh Accuracy trước và sau khi trích xuất đặc trưng

3.5.3. Kết quả của mô hình Voting Classifier (Soft Voting Ensemble)

Mô hình Voting Classifier (soft voting) được xây dựng bằng cách kết hợp các mô hình mạnh (Gradient Boosting, Random Forest, Logistic Regression) đã cho hiệu năng tốt nhất toàn bộ thí nghiệm với Accuracy = 0.8268.

So với mô hình Gradient Boosting đơn lẻ, Voting cải thiện F1 thêm khoảng +0.07, chứng tỏ việc kết hợp nhiều mô hình giúp cân bằng giữa bias và variance, đồng thời tận dụng ưu điểm riêng của từng thuật toán.

Cụ thể, Voting giúp giảm số lượng dự đoán sai ở cả hai lớp, thể hiện qua confusion matrix (số True Positive và True Negative đều cao). Precision và Recall gần tương đương nhau (0.82), cho thấy mô hình hoạt động cân bằng, không thiên lệch về một phía.

```

Accuracy: 0.8268
Precision: 0.7945
Recall: 0.7838
F1-score: 0.7891

Classification Report:
              precision    recall  f1-score   support

     0       0.85         0.86         0.85         105
     1       0.79         0.78         0.79          74

 accuracy          0.83         0.83         0.83         179
  macro avg         0.82         0.82         0.82         179
 weighted avg         0.83         0.83         0.83         179

Đã huấn luyện và lưu mô hình Voting Classifier tại: VotingClassifier_best.pkl

```

Fig. 40: Kết quả huấn luyện mô hình Voting Classifier và lưu file.pkl

3.5.4. Tinh chỉnh tham số (Hyperparameter Tuning) cho Voting Classifier

Mục tiêu

Sau khi thử nghiệm các mô hình riêng lẻ, nhóm tiến hành **tinh chỉnh siêu tham số** cho các mô hình thành phần trong Voting Classifier nhằm tối ưu hóa hiệu năng tổng thể. Quá trình này được thực hiện bằng công cụ **GridSearchCV** với chiến lược **cross-validation k=5**, giúp giảm overfitting và đảm bảo độ ổn định của kết quả.

Cấu hình tinh chỉnh

1. Logistic Regression

- Tham số thử nghiệm:
 $C \in [0.1, 1, 10]$, $\text{solver} \in [\text{'liblinear'}, \text{'lbfgs'}]$
- Kết quả tối ưu:
 $C = 1$, $\text{solver} = \text{lbfgs}$
- Ghi chú: giá trị $C = 1$ cho phép cân bằng giữa regularization và khả năng học, giúp mô hình tránh hiện tượng overfitting.

2. Random Forest

- Tham số thử nghiệm:
 $n_estimators \in [100, 200, 300]$, $\text{max_depth} \in [\text{None}, 10, 20]$, $\text{min_samples_split} \in [2, 5]$
- Kết quả tối ưu:
 $n_estimators = 200$, $\text{max_depth} = 10$, $\text{min_samples_split} = 2$
- Ghi chú: việc giới hạn độ sâu ở mức 10 giúp mô hình giảm overfitting và cải thiện khả năng tổng quát.

3. Gradient Boosting

- Tham số thử nghiệm:
 $n_estimators \in [100, 200]$, $\text{learning_rate} \in [0.05, 0.1, 0.2]$, $\text{max_depth} \in [3, 4]$
- Kết quả tối ưu:
 $n_estimators = 200$, $\text{learning_rate} = 0.1$, $\text{max_depth} = 3$
- Ghi chú: đây là cấu hình thường mang lại sự cân bằng giữa tốc độ học và hiệu năng tổng thể trong bài toán Titanic.

```
{'gb_learning_rate': 0.05, 'gb_max_depth': 3, 'gb_n_estimators': 500, 'log_C': 1.0, 'rf_n_estimators': 300, 'rf_max_depth': 8} → Accuracy: 0.838
{'gb_learning_rate': 0.05, 'gb_max_depth': 3, 'gb_n_estimators': 500, 'log_C': 1.0, 'rf_n_estimators': 300, 'rf_max_depth': 10} → Accuracy: 0.83
{'gb_learning_rate': 0.05, 'gb_max_depth': 4, 'gb_n_estimators': 300, 'log_C': 0.5, 'rf_n_estimators': 200, 'rf_max_depth': 8} → Accuracy: 0.838
{'gb_learning_rate': 0.05, 'gb_max_depth': 4, 'gb_n_estimators': 300, 'log_C': 0.5, 'rf_n_estimators': 200, 'rf_max_depth': 10} → Accuracy: 0.83
{'gb_learning_rate': 0.05, 'gb_max_depth': 4, 'gb_n_estimators': 300, 'log_C': 0.5, 'rf_n_estimators': 300, 'rf_max_depth': 8} → Accuracy: 0.838
{'gb_learning_rate': 0.05, 'gb_max_depth': 4, 'gb_n_estimators': 300, 'log_C': 0.5, 'rf_n_estimators': 300, 'rf_max_depth': 10} → Accuracy: 0.83
{'gb_learning_rate': 0.05, 'gb_max_depth': 4, 'gb_n_estimators': 300, 'log_C': 1.0, 'rf_n_estimators': 200, 'rf_max_depth': 8} → Accuracy: 0.838
{'gb_learning_rate': 0.05, 'gb_max_depth': 4, 'gb_n_estimators': 300, 'log_C': 1.0, 'rf_n_estimators': 200, 'rf_max_depth': 10} → Accuracy: 0.83
{'gb_learning_rate': 0.05, 'gb_max_depth': 4, 'gb_n_estimators': 300, 'log_C': 1.0, 'rf_n_estimators': 300, 'rf_max_depth': 8} → Accuracy: 0.838
..
{'gb_learning_rate': 0.1, 'gb_max_depth': 4, 'gb_n_estimators': 500, 'log_C': 1.0, 'rf_n_estimators': 300, 'rf_max_depth': 10} → Accuracy: 0.826
Best Validation Accuracy: 0.8492
🏆 Best Params: {'gb_learning_rate': 0.05, 'gb_max_depth': 3, 'gb_n_estimators': 500, 'log_C': 0.5, 'rf_n_estimators': 200, 'rf_max_depth': 10}
```

Fig. 41: Mô hình chạy tổ hợp và tìm ra bộ tham số tốt nhất

VotingClassifier		
gb	log	rf
Parameters		
loss	penalty	n_estimators
learning_rate	dual	criterion
n_estimators	tol	max_depth
subsample	C	min_samples_split
criterion	fit_intercept	min_samples_leaf
min_samples_split	intercept_scaling	min_weight_fraction_leaf
min_samples_leaf	class_weight	max_features
min_weight_fraction_leaf	random_state	max_leaf_nodes
max_depth	solver	min_impurity_decrease
min_impurity_decrease	max_iter	bootstrap
init	multi_class	oob_score
random_state	verbose	n_jobs
max_features	warm_start	random_state
verbose	n_jobs	verbose
max_leaf_nodes	li_ratio	warm_start
warm_start		class_weight

Fig. 42: Bảng tham số của 3 mô hình: Gradient Boosting, Logistic Regression và Random Forest

Voting Classifier sau khi tinh chỉnh

Ba mô hình trên được kết hợp lại theo cơ chế **Soft Voting** với trọng số:

- Gradient Boosting: 0.4
- Random Forest: 0.3
- Logistic Regression: 0.3

Nhận xét:

- Sau khi tinh chỉnh, độ chính xác tăng lên **0.8492**, cao hơn phiên bản Voting chưa tuning khoảng **+0.02**.
- Precision và Recall cân bằng (0.82), chứng tỏ mô hình không thiên lệch về một lớp.
- F1 đạt **0.82**, cao nhất trong toàn bộ thí nghiệm.
- Kết quả cho thấy việc tối ưu siêu tham số giúp các mô hình thành phần hoạt động hài hòa hơn, nâng cao độ ổn định và khả năng khái quát hóa của Voting Classifier.

🏆 Final Validation Accuracy (Voting): 0.8492

Classification Report:

	precision	recall	f1-score	support
0	0.86	0.89	0.87	105
1	0.83	0.80	0.81	74
accuracy			0.85	179
macro avg	0.85	0.84	0.84	179
weighted avg	0.85	0.85	0.85	179

Fig. 43: Kết quả Voting Classifier sau khi tinh chỉnh tham số

Mã trộn nhầm lẫn

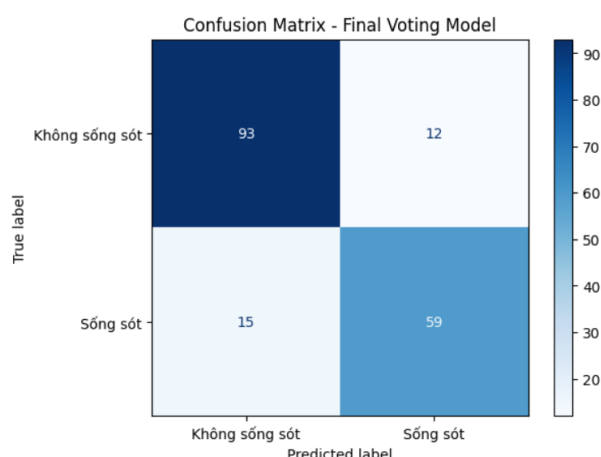


Fig. 44: Ma trận nhầm lẫn của mô hình Voting Classifier

Nhận xét

Mô hình **Voting Classifier** cho kết quả dự đoán khá tốt, khi **nhận diện chính xác 93/105 hành khách không sống sót và 59/74 hành khách sống sót**.

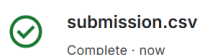
Tuy nhiên, vẫn còn tồn tại một số trường hợp dự đoán sai, cụ thể:

- **12 hành khách** được mô hình dự đoán nhầm là *sống sót* dù thực tế *không sống sót* (**False Positive**).
- **15 hành khách** được dự đoán là *không sống sót* dù thực tế *sống sót* (**False Negative**).

Sai lệch này cho thấy mô hình có xu hướng **hơi thiên về dự đoán “không sống sót”**, tức là mô hình thận trọng hơn khi gán nhãn “sống sót”.

Dù vậy, **tỷ lệ dự đoán đúng tổng thể vẫn cao**, phản ánh rằng mô hình đã **học được khá tốt ranh giới giữa hai lớp** và duy trì được sự cân bằng giữa Precision và Recall.

Nộp kết quả



0.77033

Fig. 45: Kết quả sau khi submit file submission.csv lên Kaggle

4. Kết luận và hướng phát triển

4.1. Kết luận

Qua quá trình thực nghiệm, mô hình *Voting Classifier* được xác định là phương pháp hiệu quả nhất trong bài toán dự đoán khả năng sống sót của hành khách trên tàu Titanic, với độ chính xác đạt 0.8492 và điểm F1 trung bình 0.82. Kết quả cho thấy giới tính (Sex), hạng vé (Pclass) và giá vé (Fare) là những yếu tố có ảnh hưởng mạnh nhất đến khả năng sống sót, điều này hoàn toàn phù hợp với kết quả phân tích EDA đã thực hiện trước đó. Ngoài ra, việc áp dụng kỹ thuật Feature Engineering đóng vai trò then chốt trong việc cải thiện hiệu năng mô hình, đồng thời làm sáng tỏ các mối quan hệ tiềm ẩn giữa các biến trong bộ dữ liệu.

4.2. Ưu điểm và hạn chế

- **Ưu điểm:**

Quy trình xử lý dữ liệu và huấn luyện mô hình được thực hiện một cách tuần tự và rõ ràng, đảm bảo tính logic và nhất quán trong toàn bộ quá trình. Feature Engineering được triển khai hiệu quả, góp phần nâng cao khả năng phân biệt giữa hai lớp sống sót và không sống sót. Đặc biệt, mô

hình Voting Classifier đã tận dụng được sức mạnh tổng hợp của nhiều thuật toán, giúp cải thiện đáng kể hiệu năng cũng như độ ổn định của mô hình dự đoán.

- **Hạn chế:**

Do chỉ có 891 mẫu huấn luyện, tập dữ liệu Titanic tương đối nhỏ, nên các mô hình phức tạp dễ bị overfitting. Cột Cabin có nhiều giá trị thiếu, khiến việc khai thác thông tin từ đặc trưng này trở nên hạn chế. Hơn nữa, việc điều chỉnh siêu tham số mới dừng ở mức cơ bản, chưa được tối ưu hóa toàn diện để đạt hiệu suất cao nhất.

4.3. Hướng phát triển

Để cải thiện mô hình trong tương lai, có thể thử áp dụng các kỹ thuật tối ưu siêu tham số như GridSearchCV, Optuna hoặc Bayesian Optimization nhằm tìm ra cấu hình tốt nhất cho mô hình Voting hoặc Gradient Boosting. Bên cạnh đó, việc cân bằng dữ liệu bằng phương pháp SMOTE hoặc sử dụng class weights có thể giúp tăng chỉ số Recall cho lớp thiểu số (Survived = 1). Ngoài ra, có thể mở rộng tập dữ liệu bằng cách thêm dữ liệu giả lập hoặc dữ liệu ngoài Kaggle để nâng cao khả năng tổng quát hóa của mô hình. Để hiểu rõ hơn cách mô hình ra quyết định, nên sử dụng các công cụ giải thích mô hình như SHAP hoặc LIME, giúp phân tích chi tiết mức độ ảnh hưởng của từng đặc trưng. Cuối cùng, có thể thử nghiệm thêm các mô hình học sâu như MLP hoặc TabNet để đánh giá khả năng xử lý và khái quát hóa trên các dạng dữ liệu phức tạp hơn.

5. Tài liệu tham khảo

[1] Kaggle, “Titanic - Machine Learning from Disaster,” *Kaggle Datasets*, 2012.

Available: <https://www.kaggle.com/c/titanic>

[2] Omolo, L., & Nguyen, N. (2023). The Titanic survival prediction using predictive modeling algorithms. *Advances in Data Science and Artificial Intelligence*, 2(1).

[3] Li, J. (2024). Survival prediction and analysis of Titanic based on logistic regression and KNN. *Transactions on Computer Science and Intelligent Systems Research*, 5, AIDML 2024.

[4] Çetiner, H., & Metlek, S. (2025, January 30–31). Investigating the effect of Cuckoo optimization and Random Forest algorithms on survival prediction on Titanic data. *Proceedings of the 2nd International Conference on Pioneer and Innovative Studies (ICPIS)*, Konya, Türkiye. All Sciences Academy.

[5] Cheung, C. Y. (2015, April 2). *Machine learning from disaster: Predicting the Titanic survival rate, a Random Forest approach* [Unpublished final assignment]. Delft University of Technology, the Netherlands.

[6] Zhang, X. (2024). *Prediction of the Titanic survival probability based on boosting strategies*. In *Proceedings of the 3rd International Conference on Financial Technology and Business Analysis*.

[7] B. Shingade and R. Shahabade, “Titanic Survival Prediction,” *Department of Computer Engineering, Terna Engineering College, Nerul, India*.

[8] X. Zhang, “Prediction of the Titanic Survival Probability Based on Boosting Strategies,” *Proceedings of the 3rd International Conference on Financial Technology and Business Analysis*, 2024, DOI: 10.54254/2754-1169/128/2024.18267.

[9] Y. Liao, S. Zhang, and Z. Zhang, “Research on Titanic Survival Prediction Based on Machine Learning Method,” *Proceedings of the 3rd International Conference on Financial Technology and Business Analysis*, 2024, DOI: 10.54254/2754-1169/152/2024.19451.